

UNIT 3

Inheritance and Polymorphism

The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

Inheritance is a feature or a process in which new classes are created from the existing classes. The new class created is called “derived class” or “child class” and the existing class is known as the “base class” or “parent class”. The derived class now is said to be inherited from the base class.

When we say derived class inherits the base class, it means, the derived class inherits all the properties of the base class, without changing the properties of base class and may add new features to its own. These new features in the derived class will not affect the base class. The derived class is the specialized class for the base class.

- Sub Class: The class that inherits properties from another class is called Subclass or Derived Class.
- Super Class: The class whose properties are inherited by a subclass is called Base Class or Superclass.

Base Class: A base class is a class in Object-Oriented Programming language, from which other classes are derived. The class which inherits the base class has all members of a base class as well as can also have some additional properties. The Base class members and member functions are inherited from the Object of the derived class. A base class is also called **parent class** or **superclass**.

Derived Class: A class that is created from an existing class. The derived class inherits all members and member functions of a base class. The derived class can have more functionality with respect to the Base class and can easily access the Base class. A Derived class is also called a **child class** or **subclass**.

Difference between Base Class and Derived Class:

S.No.	BASE CLASS	DERIVED CLASS
1.	A class from which properties are inherited.	A class from which is inherited from the base class.
2.	It is also known as parent class or superclass.	It is also known as the child class subclass.
3.	It cannot inherit properties and methods of Derived Class.	It can inherit properties and methods of Base Class.
4.	Syntax: class BaseClass{ // members.... // member function }	Syntax: class DerivedClass : access_specifier BaseClass{ // members.... // member function }

Syntax for creating Derived Class:

```
class BaseClass{  
    // members....  
    // member function  
}
```

```
class DerivedClass : public BaseClass{  
    // members....  
    // member function  
}
```

Where

class — keyword to create a new class

DerivedClass — name of the new class, which will inherit the base class

access-specifier — either of private, public or protected. If neither is specified, PRIVATE is taken as default

BaseClass — name of the base class

Note: A derived class doesn't inherit *access* to private data members. However, it does inherit a full parent object, which contains any private members which that class declares.

Example:

1. class ABC : private XYZ //private derivation
 { }
2. class ABC : public XYZ //public derivation
 { }
3. class ABC : protected XYZ //protected derivation
 { }
4. class ABC: XYZ //private derivation by default
 { }

Note:

When a base class is privately inherited by the derived class, public members of the base class become the private members of the derived class and therefore, the public members of the base class can only be accessed by the member functions of the derived class. They are inaccessible to the objects of the derived class.

On the other hand, when the base class is publicly inherited by the derived class, public members of the base class also become the public members of the derived class. Therefore, the public members of the base class are accessible by the objects of the derived class as well as by the member functions of the derived class.

```
// Example: define member function without argument within the class
```

```
#include <iostream>

using namespace std;

class Person {
    int id;
    char name[100];
public:
    void set_p()
    {
        cout << "Enter the Id:";
        cin >> id;
        cout << "Enter the Name:";
        cin >> name;
    }
    void display_p()
    {
        cout << endl << "Id: " << id << "\nName: " << name << endl;
    }
};
```

```
class Student : private Person {
    char course[50];
    int fee;

public:
    void set_s()
    {
        set_p();
        cout << "Enter the Course Name:";
```

```

        cin >> course;

        cout << "Enter the Course Fee:";

        cin >> fee;

    }

    void display_s()
    {
        display_p();

        cout << "Course: " << course << "\nFee: " << fee << endl;

    }

};

int main()
{
    Student s;

    s.set_s();

    s.display_s();

    return 0;

}

```

Output:

Enter the Id: 101

Enter the Name: Dev

Enter the Course Name: GCS

Enter the Course Fee:70000

Id: 101

Name: Dev

Course: GCS

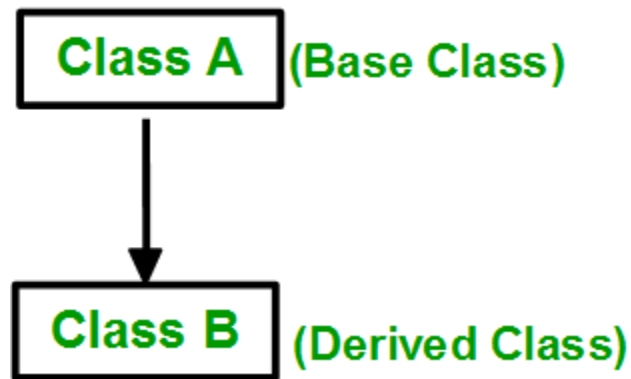
Fee: 70000

Types Of Inheritance:-

1. Single inheritance
2. Multilevel inheritance
3. Multiple inheritance
4. Hierarchical inheritance
5. Hybrid inheritance

Types of Inheritance in C++

1. Single Inheritance: In single inheritance, a class is allowed to inherit from only one class. i.e. one subclass is inherited by one base class only.



Syntax:

```
class subclass_name : access_mode base_class
```

```
{
```

```
// body of subclass
```

```
};
```

OR

```
class A
```

```
{
```

```
... ..
```

```
};
```

```
class B: public A
```

```
{... ..};
```

```
// C++ program to explain
// Single inheritance
#include<iostream>
using namespace std;

// base class
class Vehicle {
public:
    Vehicle()
    {
        cout << "This is a Vehicle\n";
    }
};

// sub class derived from a single base classes
class Car : public Vehicle {

};

// main function
int main()
{
    // Creating object of sub class will
    // invoke the constructor of base classes
    Car obj;
    return 0;
}
```

Output

This is a Vehicle

// Example:

```
#include<iostream>
using namespace std;
```

```
class A
{
    protected:
        int a;

    public:
        void set_A()
        {
            cout<<"Enter the Value of A=";
            cin>>a;
        }
        void disp_A()
        {
            cout<<endl<<"Value of A="<<a;
        }
};
```

```
class B: public A
{
    int b,p;

    public:
        void set_B()
        {
            set_A();
            cout<<"Enter the Value of B=";
            cin>>b;
        }

        void disp_B()
        {
            disp_A();
            cout<<endl<<"Value of B="<<b;
        }

        void cal_product()
        {
            p=a*b;
            cout<<endl<<"Product of "<<a<<" * "<<b<<" = "<<p;
```



```
        }  
};  
  
main()  
{  
  
    B _b;  
    _b.set_B();  
    _b.cal_product();  
  
    return 0;  
  
}
```

Output:- Enter the Value of A= 3 3 Enter the Value of B= 5 5 Product of 3 * 5 = 15

// Example:

```
#include<iostream>
using namespace std;
```

```
class A
{
    protected:
    int a;

    public:
    void set_A(int x)
    {
        a=x;
    }

    void disp_A()
    {
        cout<<endl<<"Value of A="<<a;
    }
};

class B: public A
{
    int b,p;

    public:
    void set_B(int x,int y)
    {
        set_A(x);
        b=y;
    }

    void disp_B()
    {
        disp_A();
        cout<<endl<<"Value of B="<<b;
    }

    void cal_product()
    {
        p=a*b;
        cout<<endl<<"Product of "<<a<<" * "<<b<<" = "<<p;
    }
};
```

```

main()
{
    B _b;
    _b.set_B(4,5);
    _b.cal_product();

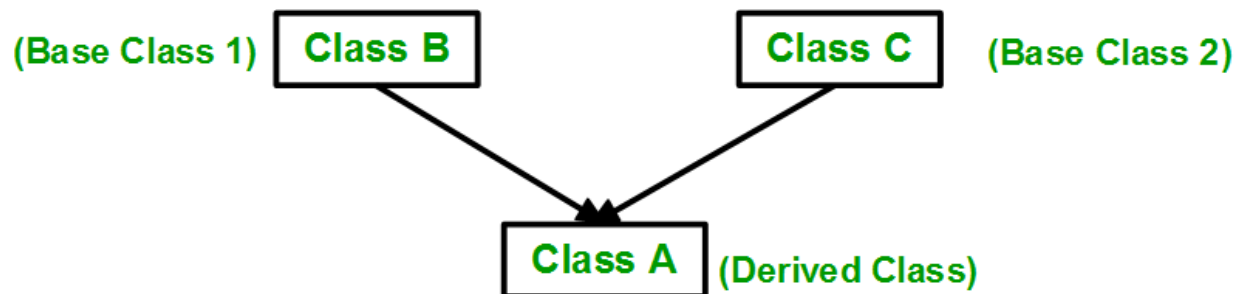
    return 0;
}

```

Output

Product of 4 * 5 = 20

2. Multiple Inheritance: Multiple Inheritance is a feature of C++ where a class can inherit from more than one class. i.e one **subclass** is inherited from more than one **base class**.



Syntax:

```

class subclass_name : access_mode base_class1, access_mode base_class2, ....
{
    // body of subclass
};

class B
{
    ... ..
};

```

```

class C
{
... ..
};

class A: public B, public C
{
... ..
};

```

Here, the number of base classes will be separated by a comma (‘, ‘) and the access mode for every base class must be specified.

```

// C++ program to explain
// multiple inheritance
#include <iostream>
using namespace std;

// first base class
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
};

// second base class
class FourWheeler {
public:
    FourWheeler()
    {
        cout << "This is a 4 wheeler Vehicle\n";
    }
};

```

```
// sub class derived from two base classes
class Car : public Vehicle, public FourWheeler {

};

// main function
int main()
{
    // Creating object of sub class will
    // invoke the constructor of base classes.
    Car obj;
    return 0;
}
```

Output

This is a Vehicle

This is a 4 wheeler Vehicle

// Example:

```
#include<iostream>
using namespace std;

class A
{
    protected:
        int a;

    public:
        void set_A()
        {
```

```

        cout<<"Enter the Value of A=";
        cin>>a;

    }

    void disp_A()
    {
        cout<<endl<<"Value of A="<<a;
    }

};

class B: public A
{
    protected:
        int b;

    public:
        void set_B()
        {
            cout<<"Enter the Value of B=";
            cin>>b;
        }

        void disp_B()
        {
            cout<<endl<<"Value of B="<<b;
        }

};

```

```

class C: public B
{
    int c,p;

    public:
        void set_C()
        {
            cout<<"Enter the Value of C=";
            cin>>c;
        }

        void disp_C()
        {
            cout<<endl<<"Value of C="<<c;
        }

        void cal_product()
        {
            p=a*b*c;
            cout<<endl<<"Product of "<<a<<" * "<<b<<" * "<<c<<" = "<<p;
        }
};

main()
{
    C _c;
    _c.set_A();
    _c.set_B();
    _c.set_C();
}

```

```
        _c.disp_A();
        _c.disp_B();
        _c.disp_C();
        _c.cal_product();

    return 0;

}

#include<iostream>
using namespace std;

class A
{
public:
    A() { cout << "A's constructor called" << endl; }
};

class B
{
public:
    B() { cout << "B's constructor called" << endl; }
};

class C: public B, public A // Note the order
{
public:
    C() { cout << "C's constructor called" << endl; }
};
```



```
int main()
{
    C c;
    return 0;
}
```

Output:

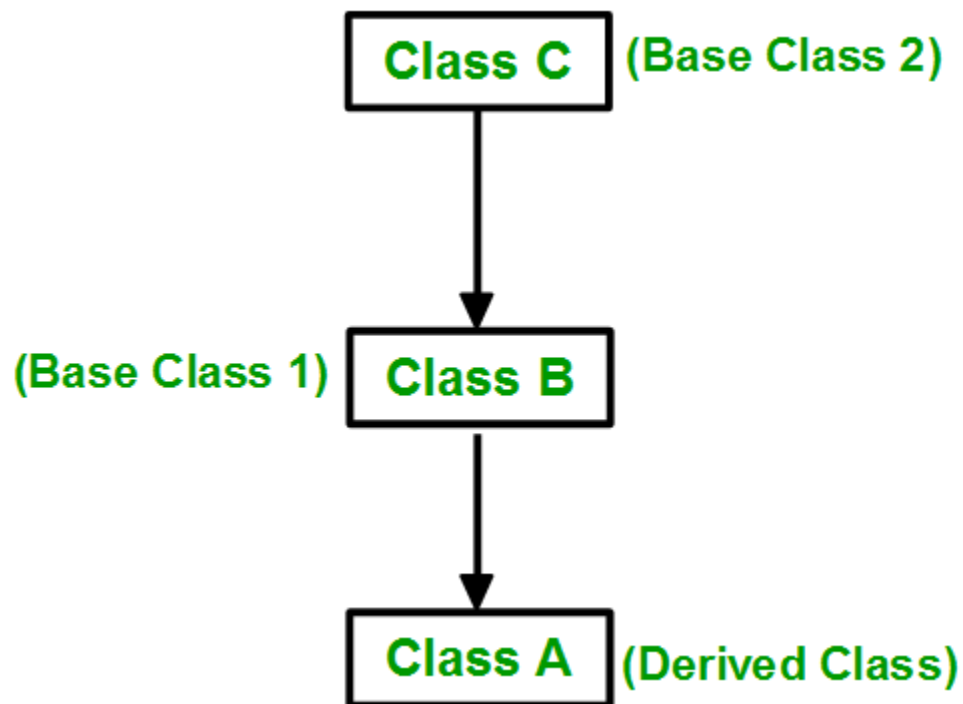
B's constructor called

A's constructor called

C's constructor called

The destructors are called in reverse order of constructors.

3. Multilevel Inheritance: In this type of inheritance, a derived class is created from another derived class.



Syntax:-

```
class C
{
    ... ..
}
```

```

};
class B:public C
{
... ..
};
class A: public B
{
... ..
};
// C++ program to implement
// Multilevel Inheritance
#include <iostream>
using namespace std;

// base class
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
};

// first sub_class derived from class vehicle
class fourWheeler : public Vehicle {
public:
    fourWheeler()
    {
        cout << "Objects with 4 wheels are vehicles\n";
    }
};

// sub class derived from the derived base class fourWheeler
class Car : public fourWheeler {

```

```

public:
    Car() { cout << "Car has 4 Wheels\n"; }
};

// main function
int main()
{
    // Creating object of sub class will
    // invoke the constructor of base classes.
    Car obj;
    return 0;
}

```

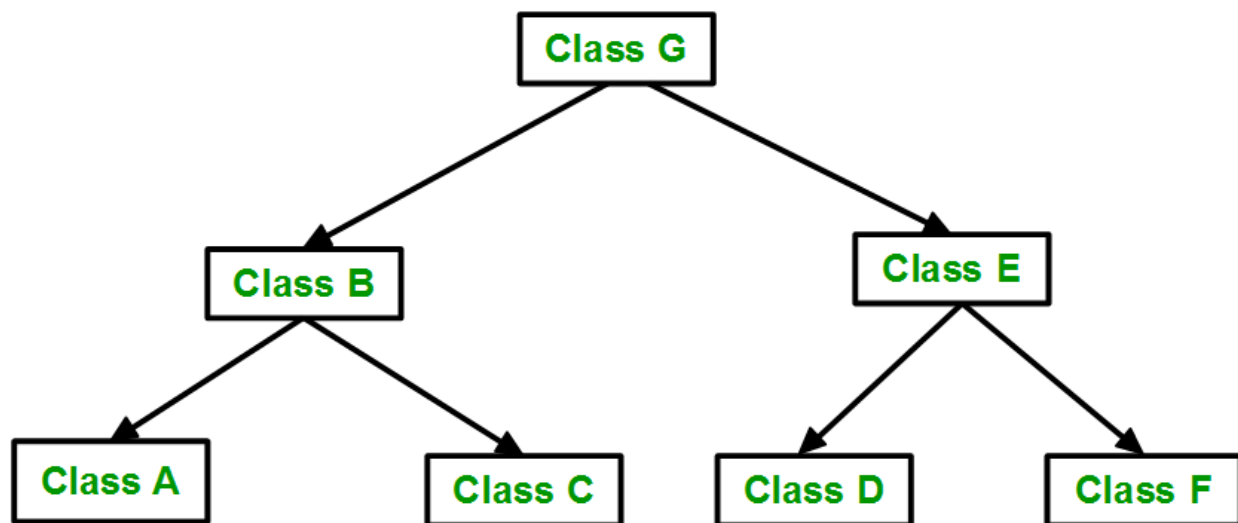
Output

This is a Vehicle

Objects with 4 wheels are vehicles

Car has 4 Wheels

4. Hierarchical Inheritance: In this type of inheritance, more than one subclass is inherited from a single base class. i.e. more than one derived class is created from a single base class.



Syntax:-

```

class A
{
    // body of the class A.
}
class B : public A
{
    // body of class B.
}
class C : public A
{
    // body of class C.
}
class D : public A
{
    // body of class D.
}

// C++ program to implement
// Hierarchical Inheritance
#include <iostream>
using namespace std;

// base class
class Vehicle {
public:
    Vehicle() { cout << "This is a Vehicle\n"; }
};

// first sub class
class Car : public Vehicle {
};

```

```
// second sub class
class Bus : public Vehicle {
};

// main function
int main()
{
    // Creating object of sub class will
    // invoke the constructor of base class.
    Car obj1;
    Bus obj2;
    return 0;
}
```

Output

This is a Vehicle

This is a Vehicle

Overriding member function

In Object-oriented programming, Inheritance is one of the most powerful concepts. It enables a class to inherit characteristics and behaviors from another class. Overriding is a technique used in C++ programming to modify the behavior of an inherited member function in a derived class.

What is Overriding Member Function?

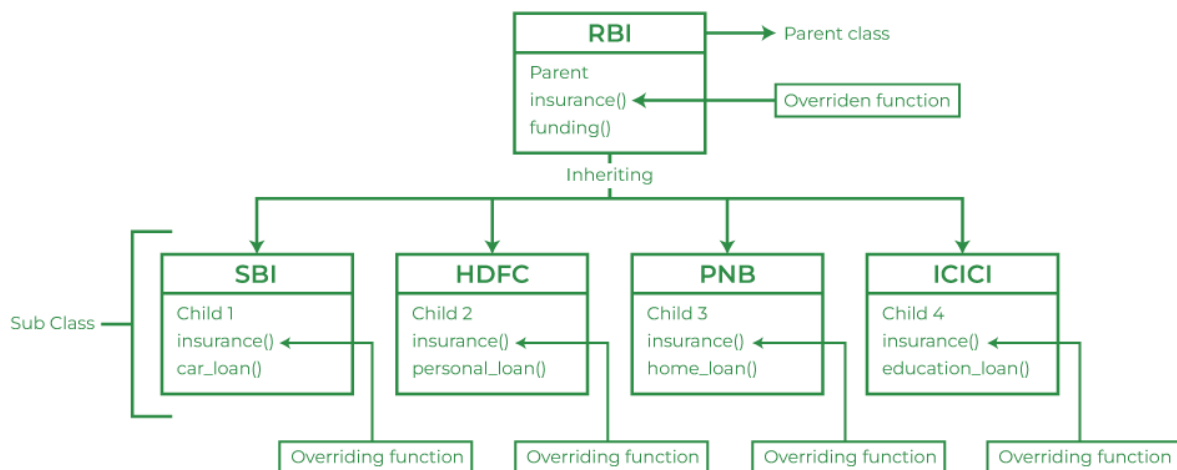
In C++, when a Derived class inherits a Member function from its Base class, it can redefine the behavior of that function in the Derived class. This process of redefining a Base class Member Function in a Derived class is called "**Overriding**" and the redefined function is referred to as an "**Overridden Member Function**".

In other words, when a Derived class defines a Member Function with the same name and signature as a Member Function in its Base class, the Derived class's function will Override the Base class function.

Real-Life Example of Function Overriding

The best Real-life example of this concept is the Constitution of India. India took the political code, structure, procedures, powers, and duties of government institutions and set out fundamental rights, directive principles, and the duties of citizens of other countries and implemented them on its own; making it the biggest constitution in the world.

Another Development real-life example could be the relationship between RBI(The Reserve Bank of India) and Other state banks like SBI, PNB, ICICI, etc. Where the RBI passes the same regulatory function and others follow it as it is.



Syntax:

```
class Parent{
```

```
    access_modifier:
```

```
        // overridden function
```

```
        return_type name_of_the_function(){}  
  
};
```

```
}
```

```
class child : public Parent {
```

```
    access_modifier:
```

```
        // overriding function
```

```
        return_type name_of_the_function(){}  
  
};
```

```
}
```

```
// C++ program to demonstrate function overriding
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Parent {
```

```
public:
```

```
    void Print()
```

```
    {
```

```
        cout << "Base Function" << endl;
```

```
    }
```

```
};
```

```
class Child : public Parent {
```

```
public:
```

```
    void Print()
```

```
    {
```

```
        cout << "Derived Function" << endl;
```

```
    }
```

```
};
```

```
int main()
```

```
{
```

```
    Child Child_Derived;
```

```
    Child_Derived.Print();
```

```
    return 0;
```

```
}
```

Output Derived Function

Variations in Function Overriding

1. Call Overridden Function From Derived Class

// C++ program to demonstrate function overriding by calling the overridden function of a member function from the child class

```
#include <iostream>
using namespace std;

class Parent {
public:
    void Print()
    {
        cout << "Base Function" << endl;
    }
};

class Child : public Parent {
public:
    void Print()
    {
        cout << "Derived Function" << endl;

        // call of overridden function
        Parent::Print();
    }
};

int main()
{
    Child Child_Derived;
    Child_Derived.Print();
    return 0;
}
```

Output

Derived Function

Base Function

2. Call Overridden Function Using Pointer

// C++ program to access overridden function using pointer of Base type that points to an object of Derived class

```
#include <iostream>
using namespace std;

class Parent {
public:
    void hello()
    {
        cout << "Base Function" << endl;
    }
};

class Child : public Parent {
public:
    void hello()
    {
        cout << "Derived Function" << endl;
    }
};

int main()
{
    Child Child_Derived;

    // pointer of Parent type that points to derived1
    Parent* ptr = &Child_Derived;

    // call function of Base class using ptr
    ptr->hello();

    return 0;
}
```

Output

Base Function

3. Access of Overridden Function to the Base Class

// C++ program to access overridden function in main() using the scope resolution operator ::

```
#include <iostream>
using namespace std;

class Parent {
public:
    void hello()
    {
        cout << "Base Function" << endl;
    }
};

class Child : public Parent {
public:
    void hello()
    {
        cout << "Derived Function" << endl;
    }
};

int main()
{
    Child Child_Derived;
    Child_Derived.hello();

    // access hello() function of the Base class
    Child_Derived.Parent::hello();
    return 0;
}
```

Output

Derived Function

Base Function

4. Access to Overridden Function

```
// C++ Program Demonstrating accessing of Overridden Function
#include <iostream>
using namespace std;

// defining of the Parent class
class Parent
{
public:
    // defining the overridden function
    void Print()
    {
        cout << "I am the Parent class function" << endl;
    }
};

// defining of the derived class
class Child : public Parent
{
public:
    // defining of the overriding function
    void Print()
    {
        cout << "I am the Child class function" << endl;
    }
};

int main()
{
    // create instances of the derived class
    Child G1, G2;

    // call the overriding function
    G1.Print();

    // call the overridden function of the Base class
    G2.Parent::Print();
    return 0;
}
```

Output

```
I am the Child class function
I am the Parent class function
```

Advantages of Overriding Member Function:

Polymorphism:

One of the major advantages of overriding is that it enables Polymorphism. Polymorphism allows a derived class to have multiple behaviors depending on the context in which it is used.

Code Reusability:

Overriding allows the reuse of code from the base class while allowing the derived class to modify the behavior of the base class's member functions.

Modularity:

Overriding makes it easier to maintain code because changes made in the derived class do not affect the base class.

Function Overloading in C++

Function overloading is a feature of object-oriented programming where two or more functions can have the same name but different parameters. When a function name is overloaded with different jobs it is called Function Overloading. In Function Overloading “Function” name should be the same and the arguments should be different. Function overloading can be considered as an example of a polymorphism feature in C++.

These functions having the same name but different arguments are known as overloaded functions. For example:

```
// same name different arguments
```

```
int test() { }
```

```
int test(int a) { }
```

```
float test(double a) { }
```

```
int test(int a, double b) { }
```

Here, all 4 functions are overloaded functions.

Notice that the return types of all these 4 functions are not the same. Overloaded functions may or may not have different return types but they must have different arguments. For example,

```
// Error code
```

```
int test(int a) { }
```

```
double test(int b){ }
```

Here, both functions have the same name, the same type, and the same number of arguments. Hence, the compiler will throw an error.

The parameters should follow any one or more than one of the following conditions for Function overloading:

- Parameters should have a different type

```
add(int a, int b)           add(double a, double b)
```

Below is the implementation of the above discussion:

```
#include <iostream>
```

```
using namespace std;
```

```
void add(int a, int b)
```

```
{
```

```
cout << "sum = " << (a + b);
```

```
}
```

```
void add(double a, double b)
```

```
{
```

```
    cout << endl << "sum = " << (a + b);
```

```
}
```

```
// Driver code
```

```
int main()
```

```
{
```

```
add(10, 2); add(5.3, 6.2); return 0;}
```

Output

```
sum = 12
```

```
sum = 11.5
```

- Parameters should have a different number

```
add(int a, int b)
```

```
add(int a, int b, int c)
```

```
#include <iostream>
```

```
using namespace std;
```

```
void add(int a, int b)
```

```
{cout << "sum = " << (a + b);}
```

```
void add(int a, int b, int c)
```

```
{cout << endl << "sum = " << (a + b + c);}
```

```
int main()
```

```
{
```

```
    add(10, 2); add(5, 6, 4); return 0;
```

```
}
```

Output sum = 12 sum = 15

- Parameters should have a different sequence of parameters.

add(int a, double b)

add(double a, int b)

```
#include<iostream>
```

```
using namespace std;
```

```
void add(int a, double b)
```

```
{ cout<<"sum = "<<(a+b);}
```

```
void add(double a, int b)
```

```
{ cout<<endl<<"sum = "<<(a+b);}
```

```
// Driver code
```

```
int main()
```

```
{
```

```
    add(10,2.5);
```

```
    add(5.5,6);
```

```
    return 0; }
```


Output sum = 12.5 sum = 11.5

Difference between Function Overloading and Function Overriding

The following are the important differences between function overloading and overriding in C++.

S.No.	Function Overloading	Function Overriding
1.	The concept through which we can define two or more functions with the same name and different numbers and parameters is known as function overloading.	The concept through which we define a function in parent class and the child class with the same return type and parameters is known as function overriding.
2.	It can take place without inheritance.	It can take place only when a class is inherited from another class.
3.	It happens during compile time.	It happens during run time.
4.	It is also known as compile time polymorphism.	It is also known as run time polymorphism.
5.	The function overloading can take place multiple times.	Function overriding can take place in the derived class only at once.
6.	Here, the scope of the overloaded functions remains the same.	Here, the scope of the overridden function is different.

- | | | |
|-----|---|--|
| 7. | No keyword is used during function overloading. | When the function is defined, it is preceded by a 'virtual' keyword in the main class. |
| 8. | The functions would be redefined with the same name, different number or type of parameters | The same function is redefined in derived class using 'out' keyword |
| 9. | Destructor can't be overloaded. | Destructor can be overridden. |
| 10. | It can be used to achieve early binding. | Overriding is also known as late binding. |

C++ Virtual Functions

A virtual function is a member function in the base class that we expect to redefine in derived classes.

Basically, a virtual function is used in the base class in order to ensure that the function is overridden. This especially applies to cases where a pointer of base class points to an object of a derived class.

For example, consider the code below:

```
class Base {  
  
    public:  
  
    void print() {  
  
        // code  
  
    }  
  
};  
  
class Derived : public Base {  
  
    public:  
  
    void print() {  
  
        // code  
  
    }  
  
};
```

Later, if we create a pointer of Base type to point to an object of Derived class and call the print() function, it calls the print() function of the Base class.

In other words, the member function of Base is not overridden.

```
int main() {  
  
    Derived derived1;  
  
    Base* base1 = &derived1;  
  
    // calls function of Base class  
  
    base1->print();  
  
    return 0;  
  
}
```

```
#include <iostream>

using namespace std;

class Base {

public:

    virtual void print() {

        cout << "Base Function" << endl;

    }

};

class Derived : public Base {

public:

    void print() {

        cout << "Derived Function" << endl;

    }

};

int main() {

    Derived derived1;

    // pointer of Base type that points to derived1

    Base* base1 = &derived1;

    // calls member function of Derived class

    base1->print();


    return 0;

}
```

Output

Derived Function

Here, we have declared the print() function of Base as virtual.

So, this function is overridden even when we use a pointer of Base type that points to the Derived object derived1.

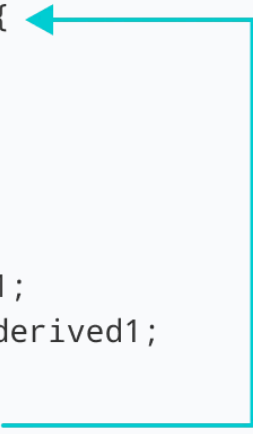
```
class Base {
    public:
        virtual void print() {
            // code
        }
};

class Derived : public Base {
    public:
        void print() {
            // code
        }
};

int main() {
    Derived derived1;
    Base* base1 = &derived1;

    base1->print();

    return 0;
}
```



print() of Derived class is called because print() of Base class is virtual

Abstract Classes in C++

Sometimes implementation of all functions cannot be provided in a base class because we don't know the implementation. Such a class is called an abstract class. For example, let Shape be a base class. We cannot provide the implementation of function draw() in Shape, but we know every derived class must have an implementation of draw().

We cannot create objects of abstract classes. However, pointers and references can be used to abstract class types.

A **pure virtual function** (or abstract function) in C++ is a virtual function for which we can have an implementation, But we must override that function in the derived class, otherwise, the derived class will also become an abstract class. A pure virtual function is declared by assigning 0 in the declaration.

Example of Pure Virtual Function

```
// An abstract class
class Test {
    // Data members of class
public:
    // Pure Virtual Function
    virtual void show() = 0;

    /* Other members */
};
```

A pure virtual function is implemented by classes that are derived from an Abstract class.

// C++ Program to illustrate the abstract class and virtual functions

```
#include <iostream>
using namespace std;
```

```
class Base {
    // private member variable
    int x;

public:
    // pure virtual function
    virtual void fun() = 0;

    // getter function to access x
    int getX() { return x; }
};
```

// This class inherits from Base and implements fun()

```
class Derived : public Base {
    // private member variable
    int y;

public:
    // implementation of the pure virtual function
    void fun() { cout << "fun() called"; }
};
```

```
int main(void)
{
    // creating an object of Derived class
    Derived d;

    // calling the fun() function of Derived class
    d.fun();

    return 0;
}
```


Output

fun() called

Some Facts

1. A class is abstract if it has at least one pure virtual function.

// C++ program to illustrate the abstract class with pure virtual functions

```
#include <iostream>
```

```
using namespace std;
```

```
class Test {
```

```
    // private member variable
```

```
    int x;
```

```
public:
```

```
    // pure virtual function
```

```
    virtual void show() = 0;
```

```
    // getter function to access x
```

```
    int getX() { return x; }
```

```
};
```

```
int main(void)
```

```
{
```

```
    Test t;
```

```
    return 0;
```

```
}
```

Write the output.

2. We can have pointers and references of abstract class type.

// C++ program that demonstrate that we can have pointers and references of abstract class type.

```
#include <iostream>

using namespace std;

class Base {
public:
    // pure virtual function
    virtual void show() = 0;
};

class Derived : public Base {
public:
    // implementation of the pure virtual function
    void show() { cout << "In Derived \n"; }
};

int main(void)
{
    // creating a pointer of type Base pointing to an object of type Derived
    Base* bp = new Derived();

    // calling the show() function using the pointer
    bp->show();

    return 0;
}
```

Write the output.

3. If we do not override the pure virtual function in the derived class, then the derived class also becomes an abstract class.

// C++ program to demonstrate that if we do not override the pure virtual function in the derived class, then the derived class also becomes an abstract class

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {
```

```
public:
```

```
    // pure virtual function
```

```
    virtual void show() = 0;
```

```
};
```

```
class Derived : public Base {
```

```
};
```

```
int main(void)
```

```
{
```

```
    // creating an object of Derived class
```

```
    Derived d;
```

```
    return 0;
```

```
}
```

Write the output.

4. An abstract class can have constructors.

// C++ program to demonstrate that an abstract class can have constructors.

```
#include <iostream>
```

```
using namespace std;
```

```
// An abstract class with constructor
```

```
class Base {
```

```
protected:
```

```
    // protected member variable
```

```
    int x;
```

```
public:
```

```
    // pure virtual function
```

```
    virtual void fun() = 0;
```

```
    // constructor of Base class
```

```
    Base(int i)
```

```
    {
```

```
        x = i;
```

```
        cout << "Constructor of base called\n";
```

```
    }
```

```
};
```

```
class Derived : public Base {
```

```
    // private member variable
```

```
    int y;
```

public:

// calling the constructor of Base class

Derived(int i, int j): Base(i)

{

 y = j;

}

// implementation of pure virtual function

void fun()

{

 cout << "x = " << x << ", y = " << y << '\n';

}

};

int main(void)

{

// creating an object of Derived class

Derived d(4, 5);

// calling the fun() function of Derived class

d.fun();

// creating an object of Derived class using a pointer of the Base class

Base* ptr = new Derived(6, 7);

// calling the fun() function using the pointer

ptr->fun(); return 0;}

Write the output.

C++ Public, Protected and Private Inheritance

In C++ inheritance, we can derive a child class from the base class in different access modes. For example,

```
class Base {
```

```
.....
```

```
};
```

```
class Derived : access_specifier Base {
```

```
.....
```

```
};
```

- public inheritance makes public members of the base class public in the derived class, and the protected members of the base class remain protected in the derived class.
- protected inheritance makes the public and protected members of the base class protected in the derived class.
- private inheritance makes the public and protected members of the base class private in the derived class.

Note: private members of the base class are inaccessible to the derived class.

```
class Base {
```

```
    public:
```

```
        int x;
```

```
    protected:
```

```
        int y;
```

```
    private:
```

```
        int z;
```

```
};
```

```
class PublicDerived: public Base {
```

```
    // x is public
```

```
    // y is protected
```

```
    // z is not accessible from PublicDerived
```

```
};
```

```
class ProtectedDerived: protected Base {
```

```
    // x is protected
```

```
    // y is protected
```

```
    // z is not accessible from ProtectedDerived
```

```
};
```

```
class PrivateDerived: private Base {
```

```
    // x is private
```

```
    // y is private
```

```
    // z is not accessible from PrivateDerived
```

```
};
```

```
// C++ program to demonstrate the working of public inheritance
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {
```

```
private:
```

```
int pvt = 1;
```

```
protected:
```

```
int prot = 2;
```

```
public:
```

```
int pub = 3;
```

```
// function to access private member
```

```
int getPVT() {
```

```
return pvt;
```

```
}
```

```
};
```



```
class PublicDerived : public Base {  
  
    public:  
  
        // function to access protected member from Base  
  
        int getProt() {  
  
            return prot;  
  
        }  
  
};  
  
int main() {  
  
    PublicDerived object1;  
  
    cout << "Private = " << object1.getPVT() << endl;  
  
    cout << "Protected = " << object1.getProt() << endl;  
  
    cout << "Public = " << object1.pub << endl;  
  
    return 0;  
  
}
```

Output

Private = 1

Protected = 2

Public = 3

Here, we have derived PublicDerived from Base in public mode.

As a result, in PublicDerived:

- prot is inherited as protected.
- pub and getPVT() are inherited as public.
- pvt is inaccessible since it is private in Base.

Since private and protected members are not accessible from main(), we need to create public functions getPVT() and getProt() to access them:

```
// Error: member "Base::pvt" is inaccessible
```

```
cout << "Private = " << object1.pvt;
```

```
// Error: member "Base::prot" is inaccessible
```

```
cout << "Protected = " << object1.prot;
```

Notice that the getPVT() function has been defined inside Base. But the getProt() function has been defined inside PublicDerived.

This is because pvt, which is private in Base, is inaccessible to PublicDerived.

However, prot is accessible to PublicDerived due to public inheritance. So, getProt() can access the protected variable from within PublicDerived.

Accessibility in public Inheritance

Accessibility	private members	protected members	public members
Base Class	Yes	Yes	Yes
Derived Class	No	Yes	Yes

// C++ program to demonstrate the working of protected inheritance

```
#include <iostream>
```

```
using namespace std;
```

```
class Base {
```

```
    private:
```

```
        int pvt = 1;
```

```
    protected:
```

```
        int prot = 2;
```

```
    public:
```

```
        int pub = 3;
```

```

// function to access private member

int getPVT() {
    return pvt;
}

};

class ProtectedDerived : protected Base {
public:
    // function to access protected member from Base
    int getProt() {
        return prot;
    }

    // function to access public member from Base
    int getPub() {
        return pub;
    }
};

int main() {
    ProtectedDerived object1;

    cout << "Private cannot be accessed." << endl;

```

```
cout << "Protected = " << object1.getProt() << endl;

cout << "Public = " << object1.getPub() << endl;

return 0;

}
```

Output

Private cannot be accessed.

Protected = 2

Public = 3

Here, we have derived ProtectedDerived from Base in protected mode.

As a result, in ProtectedDerived:

- prot, pub and getPVT() are inherited as protected.
- pvt is inaccessible since it is private in Base.

As we know, protected members cannot be directly accessed from outside the class. As a result, we cannot use getPVT() from ProtectedDerived.

That is also why we need to create the getPub() function in ProtectedDerived in order to access the pub variable.

```
// Error: member "Base::getPVT()" is inaccessible
```

```
cout << "Private = " << object1.getPVT();
```

```
// Error: member "Base::pub" is inaccessible
```

```
cout << "Public = " << object1.pub;
```

Accessibility in protected Inheritance

Accessibili ty	private members	protected members	public members
Base Class	Yes	Yes	Yes
Derived Class	No	Yes	Yes (inherited as protected variables)

// C++ program to demonstrate the working of private inheritance

```
#include <iostream>
using namespace std;
```

```
class Base {
private:
    int pvt = 1;
```

```
protected:
    int prot = 2;
```

```
public:
    int pub = 3;
```

```
    // function to access private member
    int getPVT() {
        return pvt;
    }
};
```

```
class PrivateDerived : private Base {
public:
    // function to access protected member from Base
```

```

int getProt() {
    return prot;
}

// function to access private member
int getPub() {
    return pub;
}
};

int main() {
    PrivateDerived object1;
    cout << "Private cannot be accessed." << endl;
    cout << "Protected = " << object1.getProt() << endl;
    cout << "Public = " << object1.getPub() << endl;
    return 0;
}

```

Output

```

Private cannot be accessed.
Protected = 2
Public = 3

```

Here, we have derived PrivateDerived from Base in private mode.

As a result, in PrivateDerived:

- prot, pub and getPVT() are inherited as private.
- pvt is inaccessible since it is private in Base.

As we know, private members cannot be directly accessed from outside the class. As a result, we cannot use getPVT() from PrivateDerived.

That is also why we need to create the getPub() function in PrivateDerived in order to access the pub variable.

```

// Error: member "Base::getPVT()" is inaccessible
cout << "Private = " << object1.getPVT();

```

```
// Error: member "Base::pub" is inaccessible
cout << "Public = " << object1.pub;
```

Accessibility in private Inheritance

Accessibil ity	private members	protected members	public members
Base Class	Yes	Yes	Yes
Derived Class	No	Yes (inherited as private variables)	Yes (inherited as private variables)

Ambiguity Problem in Multiple Inheritance

In Multiple Inheritance, when a single class is derived from two or more base or parent classes. So, it might be possible that both the parent class have the same-named member functions, and it shows ambiguity when the child class object invokes one of the same-named member functions. Hence, we can say, the C++ compiler is confused in selecting the member function of a class for the execution of a program.

Program to demonstrate the Ambiguity Problem in Multiple Inheritance

```
1. #include <iostream>
2. #include <conio.h>
3.
4. using namespace std;
5.
6. // create class A
7. class A
8. {
9.     public:
10.    void show()
11.    {
12.        cout << " It is the member function of class A " << endl;
13.    }
14. };
15.
16. // create class B
17. class B
18. {
19.     public:
20.    void show()
21.    {
```

```

22.     cout << " It is the member function of class B " << endl;
23. }
24. };
25.
26.
27. // create a child class to inherit the member function of class A and class B
28. class child: public A, public B
29. {
30.     public:
31.         void disp()
32.         {
33.             cout << " It is the member function of the child class " << endl;
34.         }
35. };
36.
37. int main ()
38. {
39.     // create an object of the child class to access the member function
40.     child ch;
41.     ch.show(); // It causes ambiguity
42.     ch.disp();
43.     return 0;
44. }

```

When the above program is compiled, it throws the show() member function is ambiguous.

Because of both the base class A and B, defining the same member function show(), and when the derived class's object calls the shows() function, it shows ambiguity in multiple inheritance.

Therefore, we need to resolve the ambiguous problem of multiple Inheritance. The ambiguity problem can be resolved by defining the **class name** and **scope resolution (::) operator** to specify the class from which the member function is invoked in the child class.

1. `ch.A:: show();` // class_name and scope resolution operator with member function
2. `ch.B::show();`

After making some changes, now we again execute the above program that returns the given below Output.

1. It is the member function of the child **class**
2. It is the member function of **class** A
3. It is the member function of **class** B

// C++ program to show function/method overriding

```
#include<iostream>
```

```
using namespace std;
```

```
// Base class A
```

```
class A {  
    public:  
  
    void func() {  
        cout << " I am in class A" << endl;  
    }  
};
```

```
// Base class B
```

```
class B {  
    public:  
  
    void func() {  
        cout << " I am in class B" << endl;  
    }  
};
```

```
// Derived class C
```

```
class C: public A, public B {  
    public:  
  
    // Function overriding  
    void func() {  
        cout << " I am in class C" << endl;  
    }  
};
```

```
// Driver Code
```

```
int main() {  
  
    // Created an object of class C
```

```

C obj;

// Calling function func() in class C
// through scope resolution operator
obj.C::func();

// Calling function func() in class C
// by default by compiler because of
// method or function overriding
obj.func();

return 0;
}

```

Output

```
I am in class C
```

```
I am in class C
```

In this example, we can see that we have declared function `func()` in derived class “C” also. So, when an object of class “C” is created and called the function `func()`, then, “I am in class C” is printed because it will override the base class “func” method because of function/method overriding. Now, if we do not specify which class member function “func” has to be called through the scope resolution operator then by default the compiler runs the method which is written in the body of the class through which instance of the object is created. But if the function “func” was not present in the derived class “C” then for calling function “func” in class “A” or “B”, we have to specify it through the scope resolution operator. So, the compiler will run the method of the specified class.

Virtual Function in C++

A virtual function (also known as virtual methods) is a member function that is declared within a base class and is re-defined (overridden) by a derived class. When you refer to a derived class object using a pointer or a reference to the base class, you can call a virtual function for that object and execute the derived class's version of the method.

- Virtual functions ensure that the correct function is called for an object, regardless of the type of reference (or pointer) used for the function call.
- They are mainly used to achieve Runtime polymorphism.
- Functions are declared with a **virtual** keyword in a base class.
- The resolving of a function call is done at runtime.
- A C++ virtual function is a member function in the base class that you redefine in a derived class. It is declared using the virtual keyword.
- It is used to tell the compiler to perform dynamic linkage or late binding on the function.
- There is a necessity to use the single pointer to refer to all the objects of the different classes. So, we create the pointer to the base class that refers to all the derived objects. But, when the base class pointer contains the address of the derived class object, always executes the base class function. This issue can only be resolved by using the 'virtual' function.
- A 'virtual' is a keyword preceding the normal declaration of a function.
- When the function is made virtual, C++ determines which function is to be invoked at the runtime based on the type of the object pointed by the base class pointer.

Late binding or Dynamic linkage

In late binding function calls are resolved during runtime. Therefore the compiler determines the type of object at runtime, and then binds the function call.

Rules of Virtual Function

- Virtual functions must be members of some class.
- Virtual functions cannot be static members.
- They are accessed through object pointers.
- They can be a friend of another class.
- A virtual function must be defined in the base class, even though it is not used.
- The prototypes of a virtual function of the base class and all the derived classes must be identical. If the two functions with the same name but different prototypes, C++ will consider them as the overloaded functions.
- We cannot have a virtual constructor, but we can have a virtual destructor
- Consider the situation when we don't use the virtual keyword.

```
#include <iostream>

using namespace std;

class A
{
    int x=5;

    public:
    void display()
    {
        std::cout << "Value of x is : " << x<<std::endl;
```

```

    }
};

class B: public A
{
    int y = 10;
    public:
    void display()
    {
        std::cout << "Value of y is : " << y << std::endl;
    }
};

int main()
{
    A *a;
    B b;
    a = &b;
    a->display();
    return 0;
}

```

Output:

Value of x is : 5

In the above example, * a is the base class pointer. The pointer can only access the base class members but not the members of the derived class. Although C++ permits the base pointer to point to any object derived from the base class, it cannot directly access the members of the

derived class. Therefore, there is a need for a virtual function which allows the base pointer to access the members of the derived class.

C++ virtual function Example

Let's see the simple example of C++ virtual function used to invoke the derived class in a program.

```
1. #include <iostream>
2. {
3.   public:
4.   virtual void display()
5.   {
6.     cout << "Base class is invoked"<<endl;
7.   }
8. };
9. class B:public A
10. {
11.   public:
12.   void display()
13.   {
14.     cout << "Derived Class is invoked"<<endl;
15.   }
16. };
17. int main()
18. {
```

```
19. A* a; //pointer of base class
20. B b; //object of derived class
21. a = &b;
22. a->display(); //Late Binding occurs
23. }
```

Output:

Derived Class is invoked

C++ friend Function

Data hiding is a fundamental concept of object-oriented programming. It restricts the access of private members from outside of the class.

Similarly, protected members can only be accessed by derived classes and are inaccessible from outside. For example,

```
class MyClass {

private:

    int member1;

}

int main() {

    MyClass obj;

    // Error! Cannot access private members from here.

    obj.member1 = 5;

}
```

However, there is a feature in C++ called friend functions that break this rule and allow us to access member functions from outside the class.

A friend function can access the private and protected data of a class. We declare a friend function using the friend keyword inside the body of the class.

```
class className {
```

```
... ..
```

```
    friend returnType functionName(arguments);
```

```
... ..
```

```
}
```

Characteristics of a Friend function:

- The function is not in the scope of the class to which it has been declared as a friend.
- It cannot be called using the object as it is not in the scope of that class.
- It can be invoked like a normal function without using the object.
- It cannot access the member names directly and has to use an object name and dot membership operator with the member name.
- It can be declared either in the private or the public part.

```
// C++ program to demonstrate the working of friend function
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Distance {
```

```
    private:
```

```
        int meter;
```

```
        // friend function
```

```
        friend int addFive(Distance);
```

```
    public:
```

```
        Distance() : meter(0) {}
```

```
};
```

```
// friend function definition
```

```
int addFive(Distance d) {  
  
    //accessing private members from the friend function  
  
    d.meter += 5;  
  
    return d.meter;  
  
}
```

```
int main() {  
  
    Distance D;  
  
    cout << "Distance: " << addFive(D);  
  
    return 0;  
  
}
```

Output

Distance: 5

Here, addFive() is a friend function that can access both private and public data members.

Though this example gives us an idea about the concept of a friend function, it doesn't show any meaningful use.

A more meaningful use would be operating on objects of two different classes. That's when the friend function can be very helpful.

Example when the function is friendly to two classes.

```
1. #include <iostream>
2. using namespace std;
3. class B;    // forward declarartion.
4. class A
5. {
6.     int x;
7.     public:
8.     void setdata(int i)
9.     {
10.         x=i;
11.     }
12.     friend void min(A,B);    // friend function.
13. };
14. class B
15. {
16.     int y;
17.     public:
18.     void setdata(int i)
```

```

19. {
20.     y=i;
21. }
22. friend void min(A,B);           // friend function
23. };
24. void min(A a,B b)
25. {
26.     if(a.x<=b.y)
27.         std::cout << a.x << std::endl;
28.     else
29.         std::cout << b.y << std::endl;
30. }
31. int main()
32. {
33.     A a;
34.     B b;
35.     a.setdata(10);
36.     b.setdata(20);
37.     min(a,b);
38.     return 0;
39. }

```

Output:

10

In the above example, the min() function is friendly to two classes, i.e., the min() function can access the private members of both the classes A and B.

C++ Friend class

A friend class can access both private and protected members of the class in which it has been declared as friend.

Let's see a simple example of a friend class.

```
1. #include <iostream>
2.
3. using namespace std;
4.
5. class A
6. {
7.     int x =5;
8.     friend class B;    // friend class.
9. };
10. class B
11. {
12. public:
13.     void display(A &a)
14.     {
```



```
15.     cout<<"value of x is : "<<a.x;
16. }
17. };
18. int main()
19. {
20.     A a;
21.     B b;
22.     b.display(a);
23.     return 0;
24. }
```

Output:

value of x is : 5

In the above example, class B is declared as a friend inside class A. Therefore, B is a friend of class A. Class B can access the private members of class A.

Static Function

A static function in C++ is a member function of a class that is associated with the class itself rather than with an instance or object of the class. This means that a static function can be called without creating an instance of the class.

The static keyword is used to define a static function in C++. A static function can access only static data members and other static functions of the class. It cannot access non-static data members or non-static member functions of the class.

Syntax of Static Function in C++

The syntax of a static function in C++ is similar to that of a regular member function, with the addition of the static keyword. Here is an example:

```
class MyClass {  
  
    public:  
  
    static int myStaticFunction() {  
  
        // code  
  
    }  
  
};
```

In this example, we have defined a static member function called `myStaticFunction()` in the `MyClass` class. This function can be called without creating an instance of the class, as follows:

```
int result = MyClass::myStaticFunction();
```

Note that we have used the scope resolution operator (`::`) to access the static function since it is not associated with any particular instance of the class.

Static Member Function in a class is the function that is declared as static because of which function attains certain properties as defined below:

- A static member function is independent of any object of the class.
- A static member function can be called even if no objects of the class exist.
- A static member function can also be accessed using the class name through the scope resolution operator.
- A static member function can access static data members and static member functions inside or outside of the class.
- Static member functions have a scope inside the class and cannot access the current object pointer.

- You can also use a static member function to determine how many objects of the class have been created.

The reason we need Static member function:

- Static members are frequently used to store information that is shared by all objects in a class.
- For instance, you may keep track of the quantity of newly generated objects of a specific class type using a static data member as a counter. This static data member can be increased each time an object is generated to keep track of the overall number of objects.

// C++ Program to show the working of static member functions

```
#include <iostream>
```

```
using namespace std;
```

```
class Box
```

```
{
```

```
    private:
```

```
        static int length;
```

```
        static int breadth;
```

```
        static int height;
```

```
    public:
```

```
static void print()

{

    cout << "The value of the length is: " << length << endl;

    cout << "The value of the breadth is: " << breadth << endl;

    cout << "The value of the height is: " << height << endl;

}

};
```

```
// initialize the static data members
```

```
int Box :: length = 10;
```

```
int Box :: breadth = 20;
```

```
int Box :: height = 30;
```

```
// Driver Code
```

```
int main()
```

```
{
```

```
Box b;

cout << "Static member function is called through Object name: \n" << endl;

b.print();

cout << "\nStatic member function is called through Class name: \n" << endl;

Box::print();

return 0;

}
```

Output

Static member function is called through Object name:

The value of the length is: 10

The value of the breadth is: 20

The value of the height is: 30

Static member function is called through Class name:

The value of the length is: 10

The value of the breadth is: 20

The value of the height is: 30

